

Ordenação Topológica

Dois algoritmos fundamentais para ordenar vértices de um **Grafo Acíclico Direcionado (DAG)**: abordagem via DFS e o Algoritmo de Kahn.

ALGORITMOS DE GRAFOS

Jose Raphael Perdigão

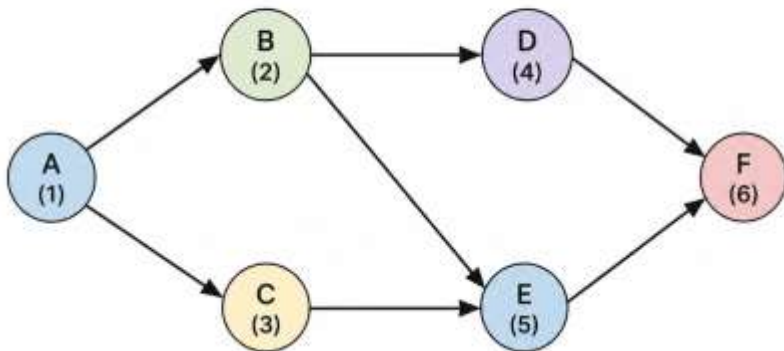
Matheus Kalleb de Souza Campos

Guilherme Teotônio Moreira

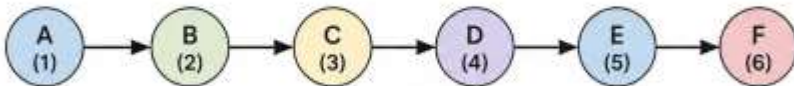


O que é Ordenação Topológica?

Dependência de Matérias



Ordem Topológica (Topological Sort)



É uma ordenação linear dos vértices de um DAG onde, para toda aresta $u \rightarrow v$, o vértice u aparece **antes** de v na sequência.

i Só é possível em grafos **sem ciclos**. Se houver ciclo, não existe ordenação válida.

Aplicações práticas

- Pré-requisitos de disciplinas
- Agendamento de tarefas com dependências
- Resolução de símbolos em compiladores

Estruturas de Dados Utilizadas

O código utiliza três estruturas estáticas para representar o grafo e gerenciar a ordem de processamento.

Grafo

Representação do grafo usando matriz de adjacência, onde $\text{matrizAdj}[u][v] = 1$ indica uma aresta de u para v . Armazena também o número total de vértices (`numVertices`).

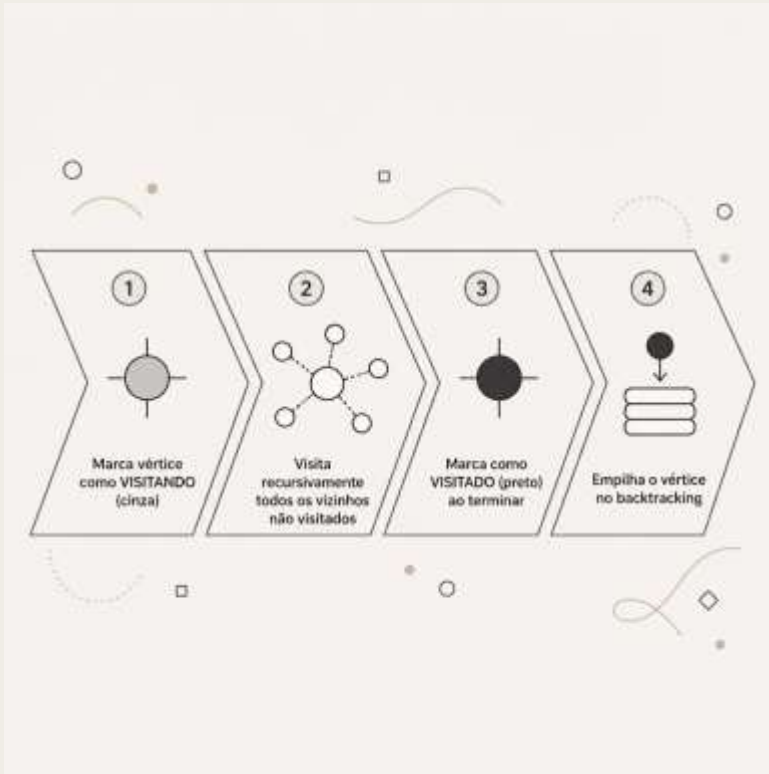
Pilha

Usada no DFS. Empilha vértices no **backtracking** para inverter a ordem pós-fixa.

Fila

Usada em Kahn. Mantém vértices com **grau de entrada zero** prontos para processamento.

Abordagem 1: DFS + Pilha



Estados dos vértices

- **NAO_VISITADO (0)** – nunca explorado
- **VISITANDO (1)** – na pilha de recursão atual
- **VISITADO (2)** – já processado completamente

Se encontrar um vértice **VISITANDO**, há um **ciclo**.

DFS: Detecção de Ciclos

O retorno 0 indica ciclo. O vértice entra na pilha **após** visitar todos os descendentes (pós-ordem).

```
int dfsAux(Grafo *g, int u, int estado[], Pilha *p) {
    if (estado[u] == VISITANDO) {return 0; // Ciclo detectado!}
    if (estado[u] == VISITADO) {return 1;}
    estado[u] = VISITANDO;
    for (int v = 0; v < g->numVertices; v++) {
        if (g->matrizAdj[u][v] == 1) {
            if (!dfsAux(g, v, estado, p)) {
                return 0; // Ciclo encontrado na recursão
            }
        }
    }
    estado[u] = VISITADO;
    push(p, u); // Empilha após visitar todos os descendentes
    return 1; // Sucesso
}

int dfsOrdenacaoTopologica(Grafo *g, int ordem[]) {
    int estado[MAX];
    Pilha p;
    int cont = 0;
    inicializarPilha(&p);

    for (int i = 0; i < g->numVertices; i++) {
        estado[i] = NAO_VISITADO;
    }
    for (int i = 0; i < g->numVertices; i++) {
        if (estado[i] == NAO_VISITADO) {
            if (!dfsAux(g, i, estado, &p)) {
                return 0; // Ciclo detectado
            }
        }
    }
    while (!pilhaVazia(&p)) {
        ordem[cont++] = pop(&p);
    }
    return 1; // Sucesso
}
```

Abordagem 2: Algoritmo de Kahn (BFS)

01

Calcular graus de entrada

Contar arestas que chegam em cada vértice.

03

Processar e liberar vizinhos

Remove da fila, adiciona à ordem e decrementa graus dos vizinhos.

02

Enfileirar grau zero

Vértices sem dependências entram na fila.

04

Verificar ciclo

Se `visitadosCont != numVertices`, há ciclo no grafo.

Kahn: Implementação

Passo 1 — Graus de entrada

```
void calcularGrausEntrada(Grafo *g, int grauEntrada[]) {
    for (int i = 0; i < g->numVertices; i++) {
        grauEntrada[i] = 0;
    }

    for (int u = 0; u < g->numVertices; u++) {
        for (int v = 0; v < g->numVertices; v++) {
            if (g->matrizAdj[u][v] == 1) {
                grauEntrada[v]++;
            }
        }
    }
}
```

Passos 2 e 3 — Fila

```
int kahn(Grafo *g, int ordem[]) {
    int grauEntrada[MAX];
    Fila f;
    int visitadosCont = 0;
    inicializarFila(&f);
    calcularGrausEntrada(g, grauEntrada);
    for (int i = 0; i < g->numVertices; i++){
        if (grauEntrada[i] == 0){
            enqueue(&f, i);
        }
    }
    while (!filaVazia(&f)) {
        int u = dequeue(&f);
        ordem[visitadosCont++] = u;
        for (int v = 0; v < g->numVertices; v++){
            if (g->matrizAdj[u][v] == 1) {
                grauEntrada[v]--;
                if (grauEntrada[v] == 0) {
                    enqueue(&f, v);
                }
            }
        }
    }
    if (visitadosCont != g->numVertices) {
        return 0; // Ciclo detectado
    }
    return 1; // Sucesso
}
```

DFS vs Kahn: Comparação



Qual escolher?

- **DFS:** mais natural se já usa recursão; detecta ciclo durante a exploração.
- **Kahn:** iterativo, evita estouro de pilha em grafos grandes; intuitivo para agendamento.

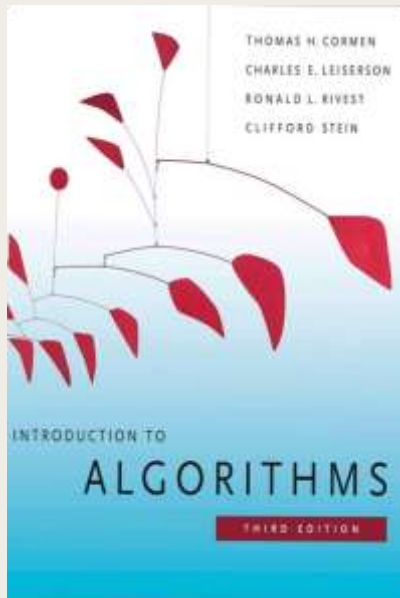


Ambos têm complexidade $O(V + E)$. A escolha depende do contexto e da estrutura do problema.

Exemplo: DAG de 6 Vértices



Referencias:



Capítulo 20:

- Busca em Profundidade (DFS)
- Ordenação Topológica



Capítulo 4.2 - Seção **Topological Sort:**

- DAGs.
- Aplicações em dependências de tarefas e disciplinas.